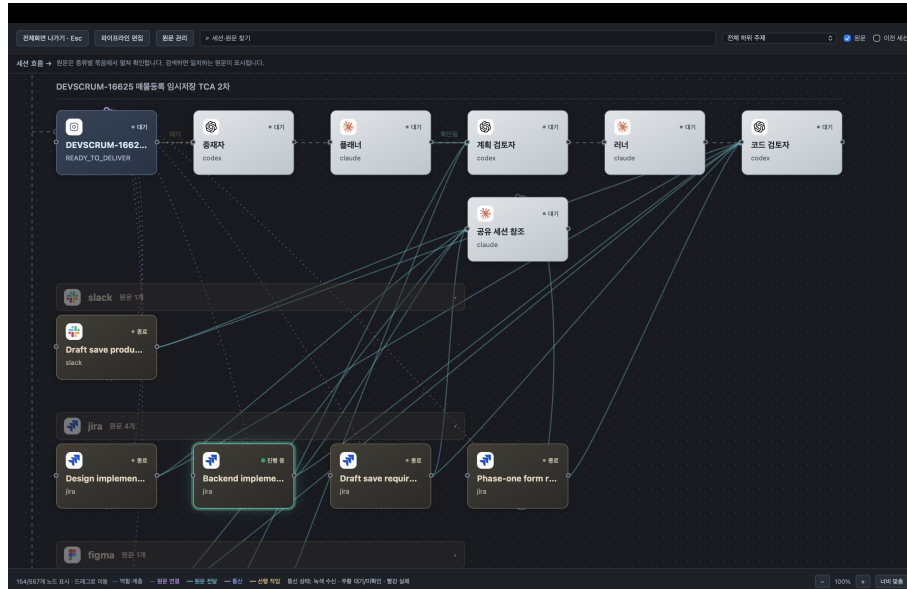


Claude·Codex 공동 개발 작업 자동화

두 AI의 계획과 구현 결과를 교차 검토하고, 수정부터 다음 작업까지 이어 가는 도구.



이 프로젝트를 시작한 계기

Codex와 Claude를 함께 사용하면서 서로 다른 강점을 확인했다. Codex는 계획 수립과 코드 검토에, Claude는 코드 작성 속도와 완성도에 강점이 있었다. 특히 Claude의 계획과 구현 결과를 Codex가 검토하면 한 세션에서 놓친 문제를 다른 세션에서 발견하는 경우가 있었다.

처음에는 두 세션의 답변을 직접 복사해 전달했다. “Claude가 이렇게 말하는데 어떻게 생각해?”라고 Codex에 묻고, 그 답변을 다시 Claude에 보내 검토를 요청했다. 검토를 반복해 두 세션의 의견이 수렴했을 때는 근거 없는 답변이 줄고 작업 결과도 좋아졌다.

리팩토링이 길어질수록 계획, 코드 변경, 리뷰 결과를 매번 복사해 전달하기가 번거로웠다. 최신 계획과 앞선 지식을 직접 대조해야 했고, 검토가 끝날 때마다 다음 작업도 요청해야 했다. 이 과정을 자동화하려고 서로의 결과에서 오류와 누락을 찾고 수정과 재검토를 반복하는 Consensus Room을 만들었다.

시스템 개요

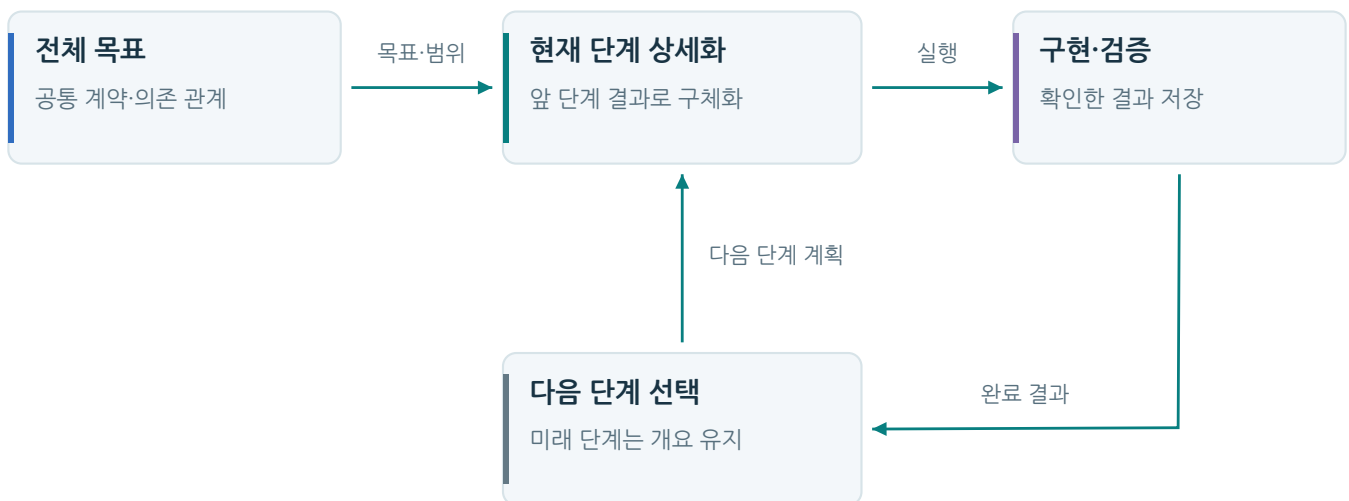
작업 엔진은 계획·구현·검토·전달을 기록하고, 승인된 계획과 실제 코드·커밋을 연결한다. 중재자는 결정과 복구를 조율한다. 웹에서는 현재 상태와 판단 근거를 확인하며, 승인된 다음 행동은 서버가 이어 간다.

세션 그래프: 역할별 참여자와 외부 원문 연결. 화면의 배경과 상태는 촬영 당시 기준.

전체 목표와 단계별 계획

전체 방향은 먼저 정하고, 실행할 단계는 앞 단계의 실제 결과를 바탕으로 구체화.

초기 흐름은 계획 단계에서 많은 요구사항과 코드를 한꺼번에 모았다. 입력이 커질 때 요약을 더 압축하거나 전달 상한을 높이는 것만으로는 해결되지 않았다. 어떤 근거가 빠졌는지 확인하기 어려웠고, 구현 시점에야 필요한 디자인 정보까지 계획에 들어갔다.



전체 계획과 현재 단계의 분리

개편에서는 전체 계획과 현재 단계의 계획을 분리했다. 전체 계획은 목표·공통 계약·단계 사이의 의존 관계를 설명한다. 세부 구현은 앞 단계의 실제 결과를 확인한 뒤 구체화한다. 이미 완료된 작업의 판단을 다음 단계마다 다시 만들지 않는 것이 목적이다.

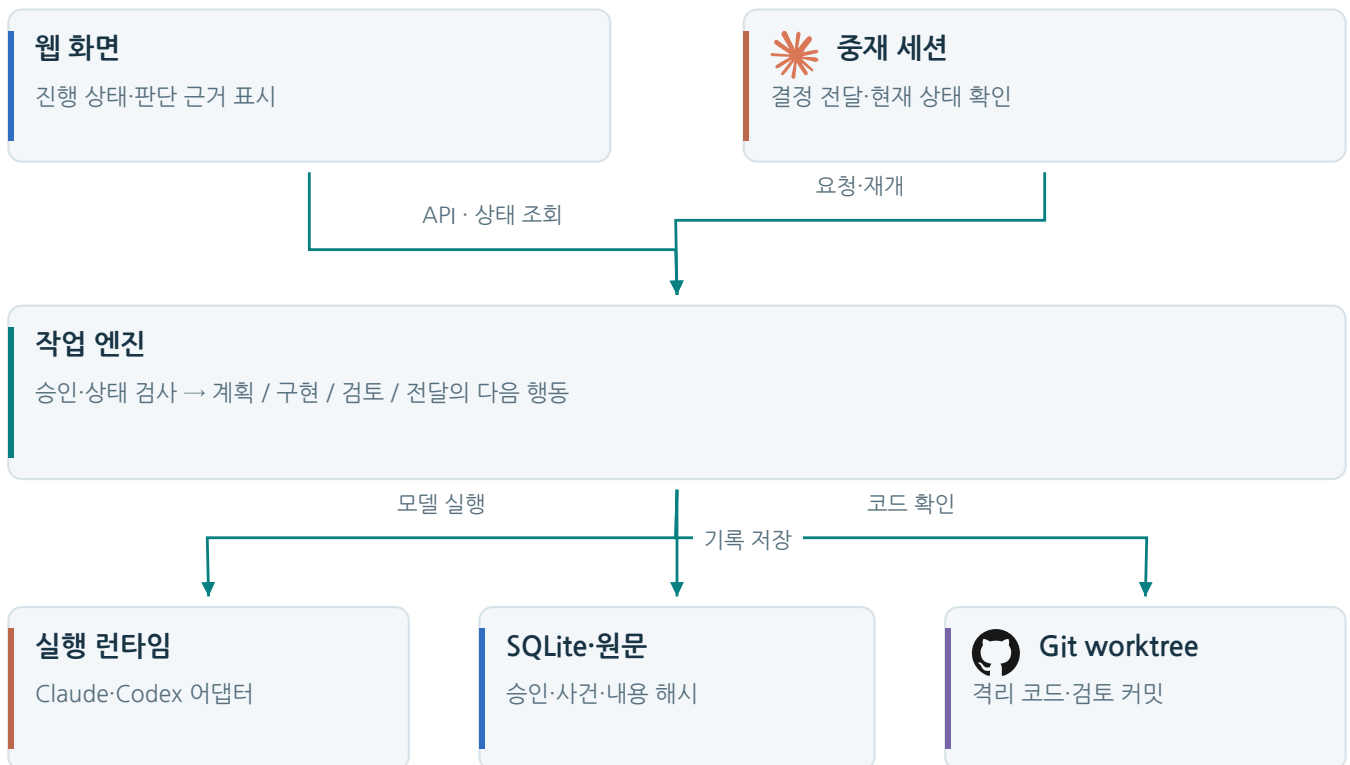
입력 크기와 작업 예산

한 번에 전달할 수 있는 크기와 전체 작업의 예산은 다른 문제다. 전자는 자료를 나눠 읽게 하는 장치이고, 후자는 비용과 실행 범위를 통제하는 장치다. 입력이 커졌다는 이유로 원문을 조용히 잘라 버리거나 새 세션으로 예산을 초기화하지 않도록 구분했다.

화면·엔진·실행 도구의 역할

상태와 권한을 판단하는 엔진, 모델을 호출하는 런타임, 사실을 보존하는 저장소.

React·Vite 대시보드는 작업·계획·리뷰·예산을 보여 준다. Node.js·TypeScript 서버는 상태 전이와 다음 행동을 결정한다. SQLite에는 승인, 역할 배정, 단계 결과, 실행 기록을 저장하고, 원문과 결과 파일은 내용 해시로 식별한다.



역할별 구현 책임

React·Vite는 화면을, Node.js·TypeScript는 서버를 구성한다. 공통 실행 런타임은 Claude·Codex CLI의 옵션과 출력 차이를 어댑터 뒤로 모은다. Git 작업 트리는 코드 변경을 격리하며, 원문과 결과 파일은 내용 해시로 식별한다.

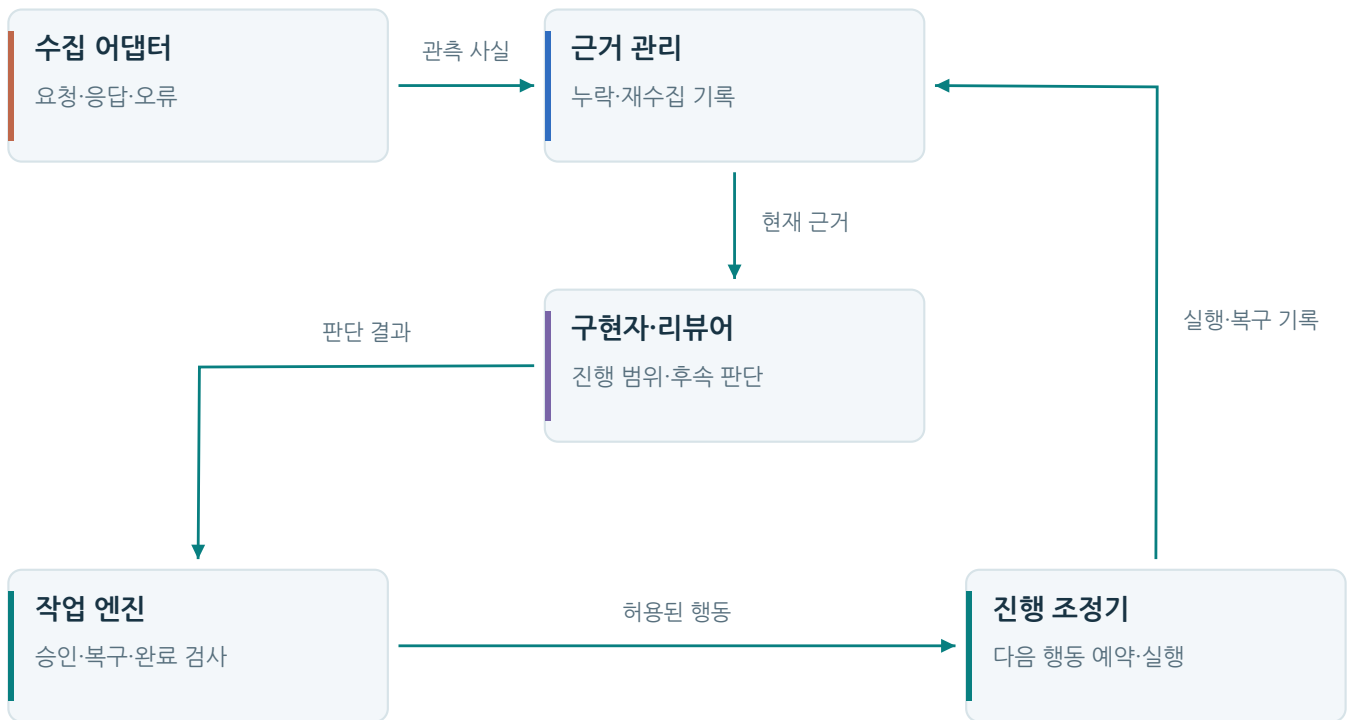
공통 실행과 정책의 경계

엔진과 호스트 리뷰는 같은 실행 계약을 쓰지만 각자의 승인과 예산을 소유한다. 런타임이 운영 DB를 대신 수정하거나 리뷰 승인을 만들어 내지 않는다. 이렇게 나누면 공급자 실행 방식을 고쳐도 작업 승인 규칙까지 바꾸지 않아도 된다.

계층별 책임과 공통 계약

수집 사실과 작업 판단을 분리하고, 다음 행동은 엔진의 공통 조건으로 실행.

자료 수집, 작업 보류, 실행 복구, 다음 단계 시작이 서로의 내부 상태를 해석하면 같은 사실을 다르게 판단할 수 있다. 자료 하나를 읽지 못한 상태가 전체 작업 중지로 번지거나, 실행을 복구해도 다음 단계는 시작되지 않는 문제가 생겼다.



수집부터 실행까지의 책임

수집 → 근거 관리 → 작업 판단 → 승인·완료 검사 → 다음 행동 실행으로 책임을 나눴다. 수집기는 관측한 사실을 저장하고, 작업 판단은 진행할 범위와 보류할 범위를 정한다. 진행 조정기는 승인 규칙을 복제하지 않고 작업 엔진의 공통 검사를 호출한다.

운영에서 드러난 계약 충돌

실제 근거 검토가 끝났는데도 결과를 받는 단계에서 실패한 적이 있었다. 읽기 전용 작업에 공통 지시가 메모리 저장 제안까지 요구했기 때문이다. 개별 오류를 허용하는 대신 작업 종류별 계약이 출력 형식·메모리 지시·검증을 함께 결정하도록 바꿨다.

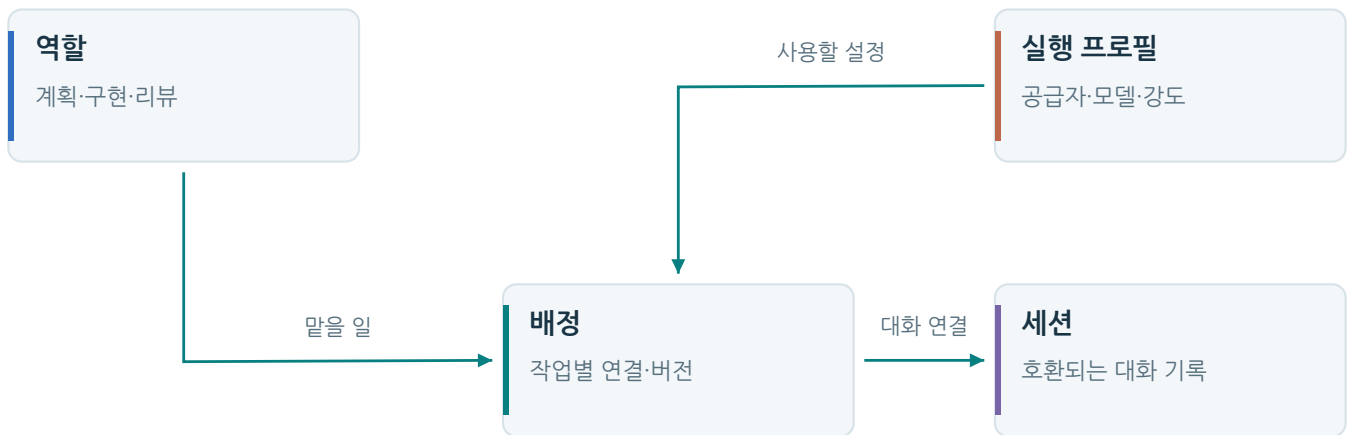
검증 전에 원본 응답과 세션·입력 위치를 보존해 같은 조사를 다시 요청하지 않고 복구할 수 있게 했다. 타입 검사에 더해 실제 API와 DB 재연결 경로에서 완료 결과가 후속 단계에 전달되는지 확인했다. 계약의 일치와 복구는 검사했으며 모델 판단의 정확성은 별도 검토 대상이다.

src/server/evidence/readLifecycle.ts · src/server/workflow.ts · src/server/engine/continuation.ts

역할·프로필·배정·세션

역할은 책임, 프로필은 실행 설정, 배정은 연결 관계, 세션은 대화 기록.

설계자·구현자·검토자라는 역할은 허용할 행동과 결과 형식을 설명한다. 실행 프로필은 공급자·모델·추론 강도·공급자별 옵션을 담는다. 배정은 어느 작업에서 어떤 프로필을 쓰는지와 그 버전을 연결한다. 세션은 해당 배정의 대화 기록이다.



배정 변경과 호환성

이 구분이 없으면 중재자를 Claude에서 Codex로 바꿀 때 역할과 무관한 설정까지 다시 만들어야 한다. 개편에서는 배정과 기능 지원 여부를 먼저 확인한 뒤 공통 실행 요청으로 변환한다. 전역 배정을 작업별 배정으로 덮어쓸 수 있지만, 기록 없이 설정을 바꾸지는 않는다.

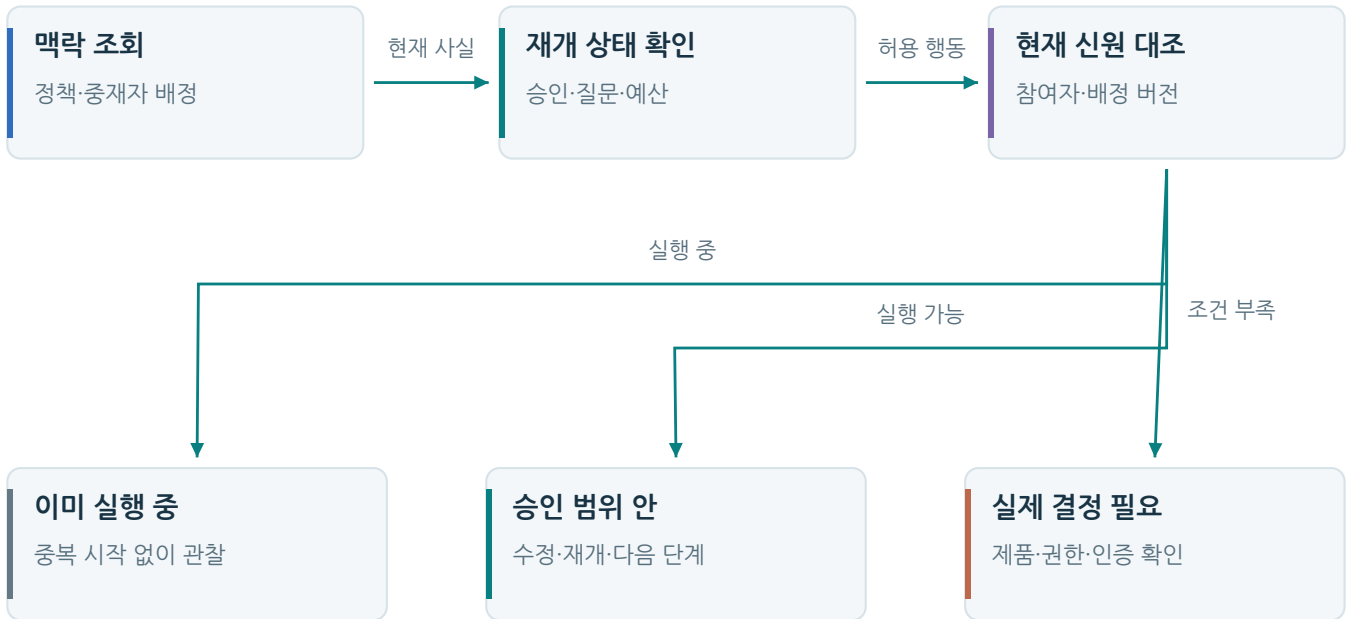
새 공급자의 연결 조건

실제 어댑터는 Claude와 Codex 두 개다. 임의의 AI 도구를 연결하면 곧바로 동작하는 구조는 아니다. 새 공급자는 실행·권한·세션·사용량·결과 파싱 계약을 구현해야 한다. ultracode 같은 옵션도 공급자별 계약으로 남겨 공통 추론 강도와 혼동하지 않는다.

중재자의 확인 순서

대화의 마지막 문장보다 서버의 현재 상태·허용 행동·배정 신원을 먼저 확인.

중재자는 어떤 작업이 진행 중인지, 무엇 때문에 멈췄는지, 남은 질문과 예산이 무엇인지 확인한다. 이미 승인된 범위 안의 수정과 다음 준비된 단계를 조율하고, 서버에 저장된 자동 진행 상태를 확인한다. 제품 결정·인증·권한·예산처럼 실제로 부족한 조건은 이유를 남기고 요청한다.



중재자 교체와 상태 확인

대화의 마지막 문장만 보고 재개하지 않도록 현재 맥락 조회, 재개 상태 확인, 역할 배정 신원 확인 순서를 둔다. 이전 승인과 질문을 유지하고 이미 실행 중인 작업을 중복 시작하지 않게 하는 절차다.

하위 작업의 연결 누락과 복구

상위 작업에는 중재 세션이 연결돼 있었지만 새 하위 작업은 미배정으로 표시됐다. 배정 조회가 자기 작업과 전역 설정만 확인해 상위 연결을 놓친 것이 원인이었다. 가장 가까운 상위 중재자 배정을 조회하되 원래 버전과 세션을 유지하도록 고쳤다.

배정을 새로 만들면 기존 신원과 승인 기록이 어긋날 수 있어 조회 규칙을 수정했다. 하위 작업에 명시한 배정은 우선한다. 공개 API에서 두 단계 상속, 서버 재시작 뒤 연결 유지, 하위 재배포 뒤 이전 신원의 거부를 확인했다.

서버가 맡는 연속 실행

승인된 행동을 DB에 저장하고, 대화의 알림 수신 여부와 별도로 실행 결과를 추적.

단계가 끝났다는 알림만 보내면 중재 대화가 응답하지 않을 때 작업도 멈춘다. 서버의 진행 조정기는 현재 계획·범위·중재자·허용된 전달 범위를 저장하고, 작업 엔진의 공통 승인·완료 검사를 거쳐 다음 행동을 실행한다.



계획 합의 뒤 멈춘 실행 경로

계획 합의는 끝났지만 승인은 근거 영향 검토를 기다렸고, 자동 검토는 첫 계획의 승인 대기를 처리하지 않았다. 알림만 보내서는 채워지지 않는 공백이었다. 계획 마무리에 근거 검토를 연결하고 다음 행동의 책임을 서버로 모았다.

승인된 계획과 재개 위치의 보존

서버는 현재 계획·범위·중재자·허용 전달 범위를 저장하고 공통 승인·완료 검사를 호출한다. 근거 검토부터 구현·리뷰·로컬 완료·다음 단계의 계획 시작까지 이어 가되 새 계획의 승인은 별도로 받는다. 실행 상태와 사건도 함께 저장해 중간 종료 뒤 재개 위치가 달라지지 않게 했다.

실제 API 검사에서는 한 번의 진행 요청 뒤 중간 행동을 대신 호출하지 않고 DB 재연결부터 다음 계획 시작까지 관찰했다. 모델 응답은 대역을 사용했다. 사용자 중지·새 계획·중재자 교체 시에는 늦은 응답으로 실행이 다시 시작되지 않는지도 확인했다.

완료 결과를 기준으로 한 단계 분리

같은 결과를 만드는 설계·구현·검사는 한 단계 안의 체크리스트로 구성.

설계·구현·테스트를 각각 별도 단계로 만들거나 파일별로 단계를 나누면 같은 내용을 여러 번 설명하고 검토하게 된다. 개편에서는 한 결과를 완성하는 작업을 단계 안의 체크리스트로 묶는다. “이 단계를 끝내면 무엇을 확인할 수 있는가?”를 먼저 적는다.



단계 분리의 판단 기준

분리는 앞 단계의 결과가 있어야 판단할 수 있는 경우, 독립적인 검증이 필요한 경우, 실패 시 따로 되돌릴 필요가 있는 경우로 설명한다. 단계 수나 코드 줄 수를 일률적으로 제한하지 않는다.

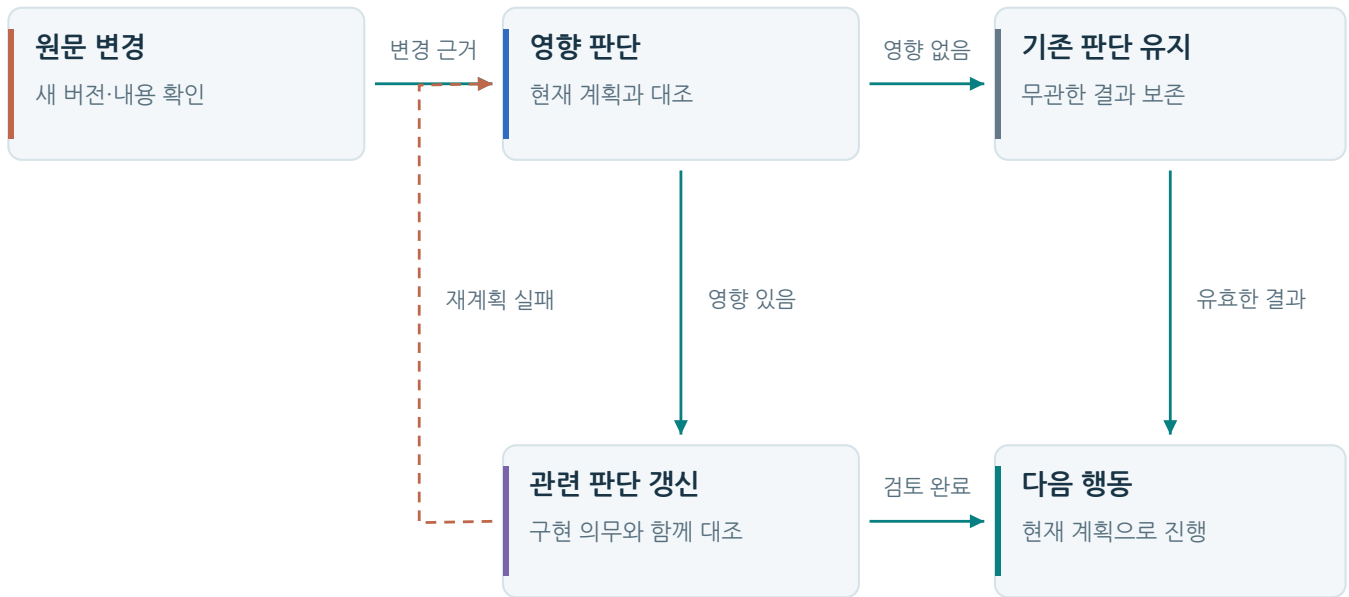
현재 단계의 완료 조건

현재 단계에는 완료 조건, 검사 방법, 예산, 해결해야 할 질문이 필요하다. 미래 단계는 개요로 남길 수 있다. 기본적으로 한 단계를 실행하며, 외부 정보나 사용자 결정을 기다릴 때만 의존하지 않는 준비된 단계로 진행할 수 있다. 예산 소진을 기다림으로 바꿔 다른 실행을 시작하지 않는다.

근거 변경과 판단 보존

변경된 근거의 영향만 다시 확인하고, 관련 없는 결과와 기존 구현 의무는 보존.

미래 단계는 추가·분할·병합할 수 있지만 완료 결과의 의미를 덮어쓰지는 않는다. 진행 중 전제가 바뀌면 변경 기록과 재계획을 연결하고, 이미 조사한 사실 중 무엇이 여전히 유효한지 구분한다.



전체 초기화에서 영향 범위 확인으로

외부 원문 하나만 바뀌어도 채택한 사실을 모두 비워, 관계없는 자료까지 다시 조사하는 문제가 있었다. 원문이 바뀐 사실과 여러 원문을 함께 인용한 결론은 다시 확인하되, 참조가 모두 유효한 독립 사실은 보존하도록 바꿨다.

사용자 지시·코드·선택한 메모리까지 갈 때만 이 재사용을 허용한다. 승인과 최종 결론은 새 근거의 모순 여부를 검토해야 하므로 별도로 갱신한다. 앞선 판단을 통째로 유지하는 것과 유효한 조사 결과를 재사용하는 것을 구분한 선택이다.

재사용의 효과와 실패 복구

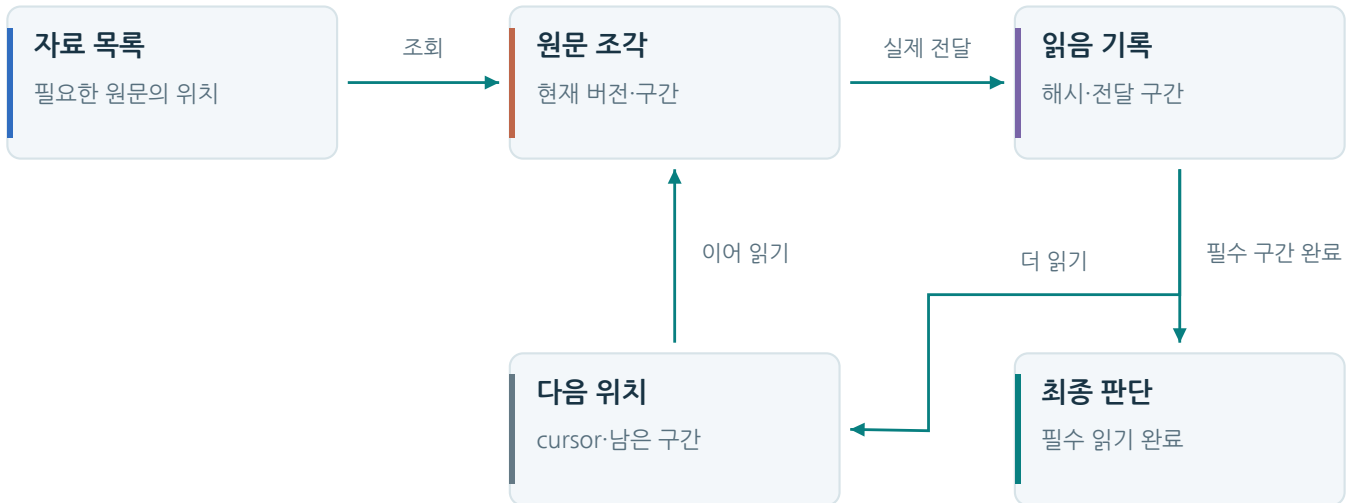
두 독립 원문 중 하나만 바꾸는 검사에서 변경 원문을 인용한 결론은 제거되고 다른 사실은 유지됐다. 기존 읽음 기록과 해시를 비교하므로 보존 여부를 확인하려고 모델을 추가 호출하지 않는다. 운영 토큰 절감률은 아직 같은 조건으로 비교하지 않았다.

재계획이 실패하면 변경과 재개 위치를 남긴다. 이전 실행의 늦은 응답은 새 계획의 결과로 받지 않으며 기존 기록과 사용량에서 이어 간다.

원문의 분할 조회와 읽음 기록

자료의 출처·버전·위치와 실제 전달 구간을 함께 보존.

계획 시작 시 목표, 진행 기록, 자료 목록과 참조를 전달한다. AI는 서버가 관리하는 읽기 도구로 파일·Git 변경·저장 자료를 조회한다. 전체 폴더와 대화 이력을 한꺼번에 넣는 대신 현재 질문을 해결하는 데 필요한 원문을 요청한다.



단계가 바뀔 때 반복된 공통 계약 읽기

10월 운영에서 근거 검토 뒤 계획 개정에 2,559초가 걸렸다. 기록에는 64,434바이트 요청서를 처음부터 다시 읽은 흔적이 있었다. 같은 공통 계약을 단계별 지시 앞에 붙여 문서 전체 해시를 바꾼 것이 원인이었다.

공통 계약을 별도 필수 문서로 분리하고 세션·작업·본문 해시·읽은 구간이 같으면 완독 기록을 재사용하도록 했다. 새 단계의 지시와 출력 형식은 현재 요청에 남겼다. 필요한 지시까지 늦추지 않으면서 반복 전송을 줄이기 위한 선택이다.

재전송 감소와 안전한 재열람

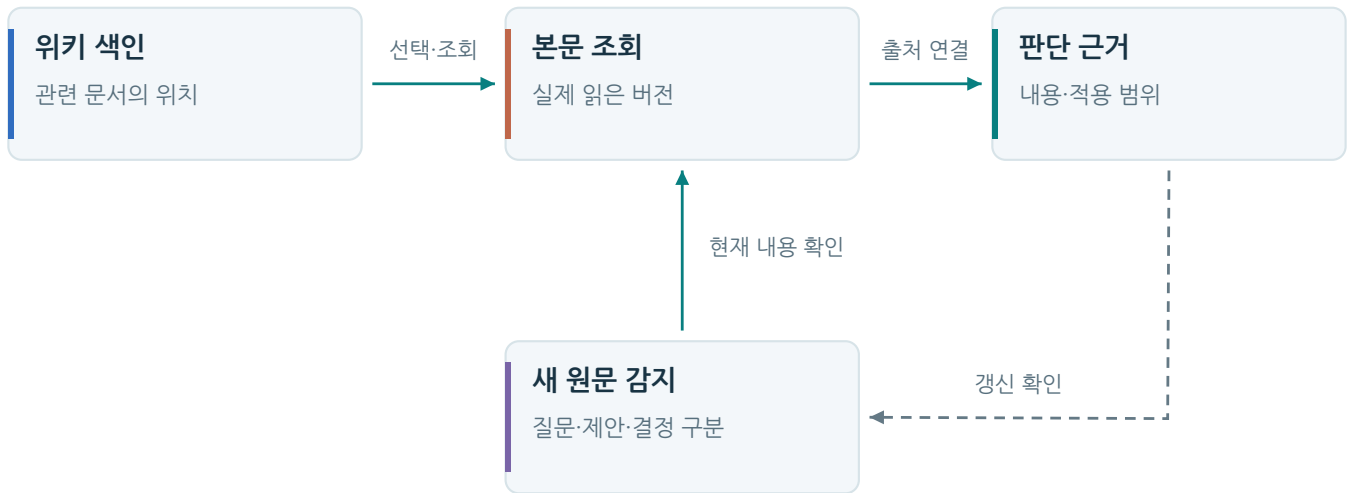
모델 대역을 사용한 실제 어댑터 검사에서 완독한 계약이 다음 개정 요청에 다시 실리지 않았다. 첫 전달도 조각마다 호출하지 않고 요청의 남은 공간에 여러 조각을 함께 넣었다. 내용 변경·세션 손실·응답 미확인 때는 다시 읽고, 미완독 구간은 계속 추적한다.

이 결과는 재전송과 불필요한 호출 경로를 줄였다는 근거다. 2,559초는 수정 전 관측이며, 수정 후 운영 시간과 토큰의 절감률은 별도 비교가 필요하다.

위키 검색과 실제 근거의 구분

색인에서 찾은 문서와 실제로 읽고 판단에 사용한 원문을 구분.

위키 색인에는 문서의 위치와 관련 주제가 있다. 처음에는 관련 문서 일부만 선택하고, 더 필요한 자료를 질의해 읽는다. 모든 위키를 프롬프트에 붙이거나 색인에 나타난 모든 문서를 판단 근거로 등록하지 않는다.



실제 읽은 문서의 기록

실제로 조회한 문서의 경로·버전·내용을 기록한다. 읽지 않은 문서가 바뀌었다는 이유로 현재 작업을 모두 무효화하지 않는다. 반대로 읽은 원문이 바뀌면 관련 판단을 다시 확인해야 한다.

새 원문의 해석과 검색 비용

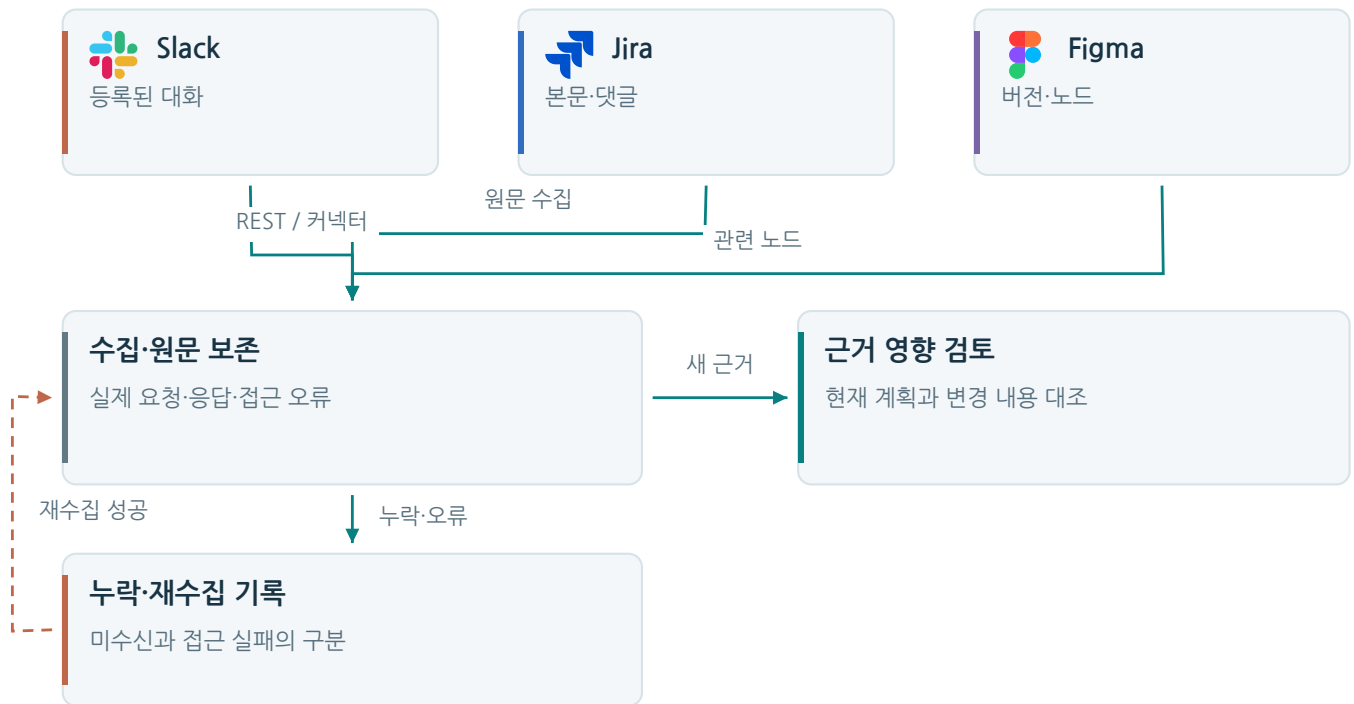
위키 본문의 해시와 외부 원문의 해시는 다른 정보를 가리킨다. 외부 자료가 갱신되면 “확인이 필요하다”는 상태를 남긴다. 새 내용이 질문인지 결정인지, 어느 플랫폼에 적용되는지, 기존 근거와 충돌하는지 판단한 뒤 결론을 고쳐야 한다.

검색 정확도가 좋아졌다고 토큰이 줄었다고 말할 수는 없다. 더 관련 있는 문서를 많이 선택하면 전달량이 늘 수도 있다. 위키 수정 제안도 경로·상태·내용 해시와 역할 권한을 확인한 뒤 반영하며, 폴더 전체를 임의로 고칠 권한을 주지 않는다.

Slack·Jira·Figma 자료 흐름

등록한 출처의 변경 감지와 모델에 전달할 근거의 판단을 분리.

Slack·Jira·Figma 링크를 등록하면 REST 수집기가 기본 15분 간격으로 조회 시점을 확인한다. 시작할 때 밀린 조회도 확인한다. 닫힌 작업에서만 사용하는 출처는 정기 조회 대상에서 빠진다. 외부 자료가 없는 작업에는 조회할 대상이 없다.



수집과 모델 전달의 분리

수집 자체는 LLM을 호출하지 않는다. 변경분만 실행 전 영향 검토와 시작·재개 입력에 연결한다. Figma는 계획에서 화면 링크를 보관하고, 구현자가 화면을 만들 때 필요한 노드와 이미지를 직접 확인한다. 링크나 캐시 이미지만으로 디자인 검증을 완료했다고 판단하지 않는다.

재확인 기한과 접근 실패의 구분

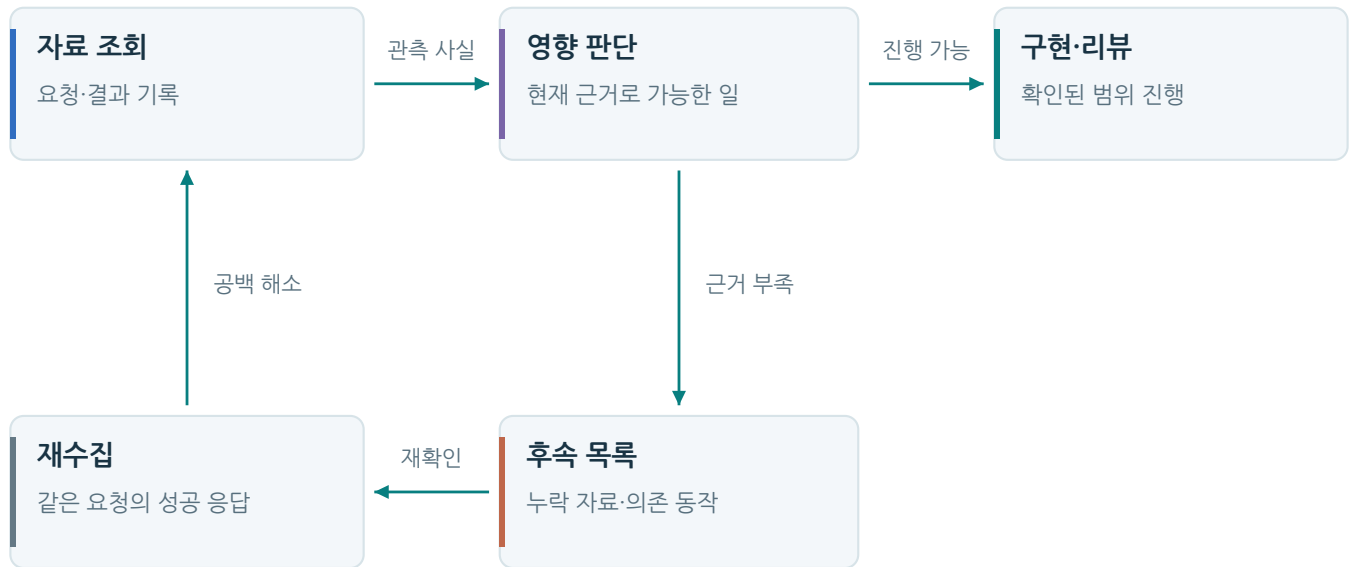
운영 중 정상 수집된 원문이 30분 뒤 읽기 대상에서 빠져 기존 판단까지 사라졌다. 직접 조회는 성공했는데 재확인 기한을 접근 실패로 해석한 것이 원인이었다. 저장된 원문의 사용 가능 여부와 최신 확인 필요 상태를 분리했다.

기한이 지나도 정상 원문은 유지하고 재확인 요청을 남긴다. 실제 접근 오류·누락은 성공으로 바꾸지 않는다. 계획 도중 30분을 넘기는 검사에서 같은 원문과 사실을 유지한 채 완료했고, 실제 수집 실패를 거부하는 검사도 유지했다.

자료 누락과 후속 작업

부족한 자료에 의존하는 부분만 후속 목록으로 넘기고, 확보한 근거의 범위에서 진행.

외부 자료에 접근할 수 없다고 확인된 요구사항까지 모두 지우면 작업이 반복해서 멈춘다. 읽지 못한 자료와 그 자료가 필요한 동작을 후속 목록에 남기고, 확보한 근거로 진행할 수 있는 범위를 구현자와 리뷰어가 함께 확인한다.



한 자료의 실패가 전체 중지로 번진 사례

운영 재개 텀에는 미수신 요청이 36개 있었다. Figma 한 요청이 호출 제한을 받아 나머지 35개를 읽지 못한 채 전체 실행이 멈췄다. 일반 정책은 부족한 부분을 보류하도록 했지만 디자인 도구는 이전 요청 전체의 재조회를 요구했다.

관측 상태와 작업 처분의 분리

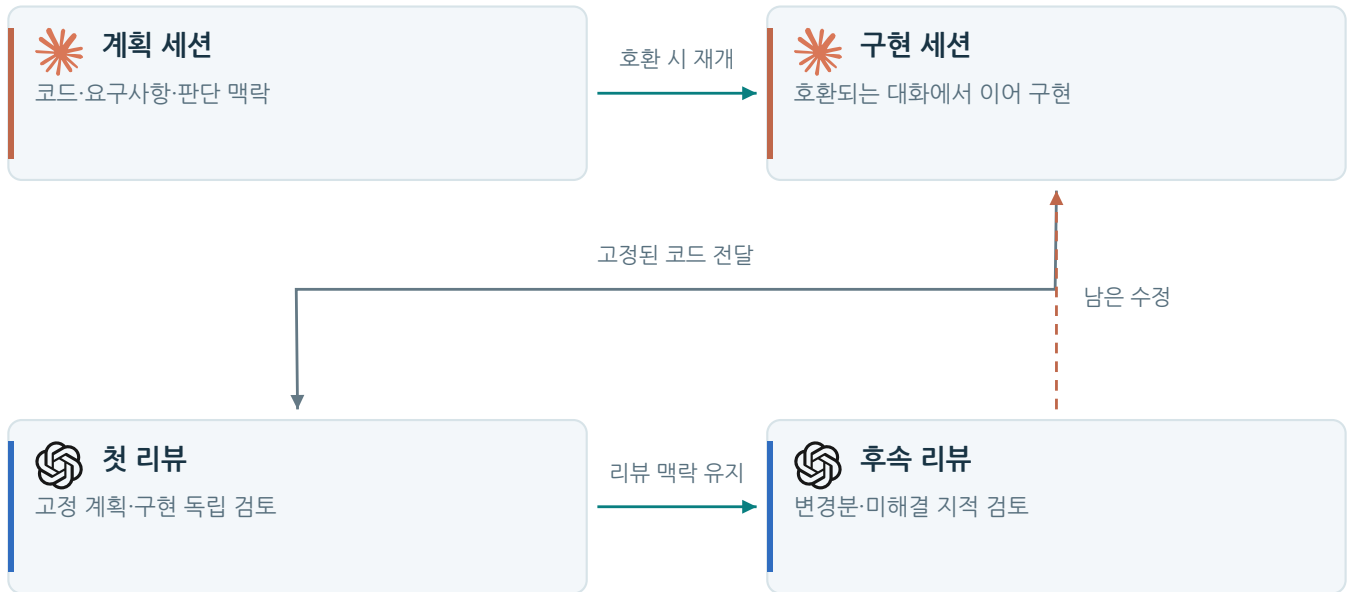
응답 미수신, 실제 접근 오류, 성공 관측을 구분하고, 그 자료가 필요한 동작만 후속 목록으로 보낸다. 구현자와 리뷰어가 같은 목록을 읽도록 해 도구별 판단이 어긋나지 않게 했다. 새 읽기 요청의 성공 응답을 확인해야 공백이 해소된다.

실제 Git·API·DB 재연결 검사에서 한 요청의 오류가 있어도 지원되는 코드 변경·리뷰·로컬 커밋·다음 계획으로 이어졌다. 나머지 미수신 35개는 후속 목록에 남았다. 모델 응답은 대역이며, 이 검사는 누락을 숨기지 않으면서 실행 경로를 이어 가는지를 확인한다.

계획·구현·리뷰의 세션 연결

호환되는 대화와 유효한 읽음 기록을 이어 사용하고, 검토자의 판단 맥락은 별도로 유지.

설계자가 이미 코드와 요구사항을 이해했는데 구현을 새 세션에서 시작하면 판단 근거를 다시 요약하고 전달해야 한다. 개편에서는 계획과 구현의 배정이 호환될 때 같은 대화를 이어 사용한다. 원문 전체를 다시 붙이는 별도 전달 문서를 필수로 만들지 않는다.



대화를 이어 쓰는 조건

이 연결은 세션 이름만으로 결정하지 않는다. 공급자, 참여자, 역할 배정과 실행 조건을 확인한다. 호환되지 않는 세션을 억지로 재개하거나 임의의 다른 모델로 대체하지 않는다.

검토자의 독립성과 재사용 범위

검토자는 고정된 계획과 구현 결과를 독립적으로 본다. 첫 리뷰는 전체 변경을 확인하고, 수정 후에는 같은 리뷰 맥락에서 바뀐 부분과 미해결 지적을 확인한다. 범위를 넓혀야 한다면 새 반증이나 공통 계약의 영향처럼 이유가 있어야 한다.

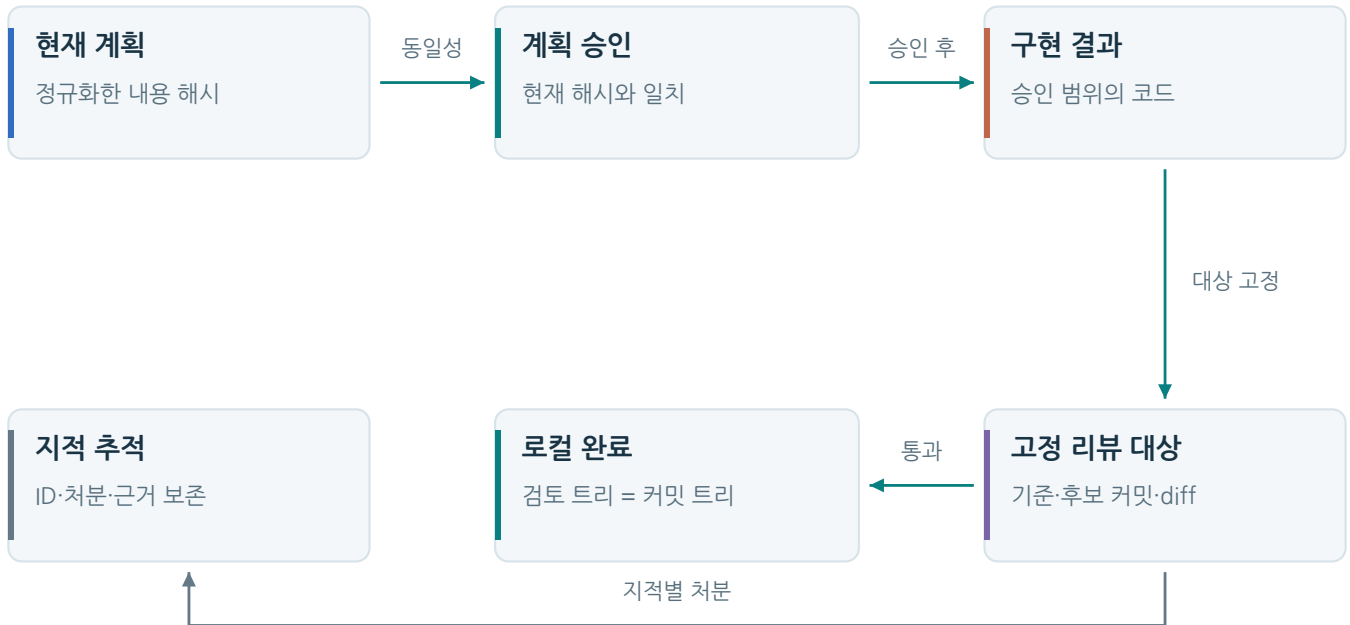
실패·취소된 전달은 읽음으로 기록하지 않는다. 사용량 확인 때문에 보류한 응답은 같은 입력·세션·원문 버전과 전달 구간을 대조해 재사용한다. 새 결정이나 원문 변경이 있으면 다시 확인한다. 실제 토큰 절감률은 같은 조건의 비교 실행으로 측정해야 한다.

아이콘은 예시 배정. 역할별 공급자는 변경 가능.

승인 계획과 검토 코드의 동일성

계획 내용, 리뷰 대상, 실제 결과를 각각 고정하고 같은 대상인지 대조.

설계자와 검토자가 서로 다른 계획을 보고 동의하는 일을 막기 위해 정규화한 계획의 SHA-256을 사용한다. 현재 승인도 그 계획에 연결한다. 해시가 같다는 사실은 같은 내용을 가리킨다는 뜻이며, 계획이 옳다는 증명은 아니다. 타당성은 별도 검토가 맡는다.



계획 동일성과 고정된 검토 대상

기준 커밋·후보 커밋·파일 목록·변경 내용을 고정한 뒤 검토한다. 수정 중인 폴더를 동시에 검토해 승인 대상이 달라지는 일을 막기 위해서다. 지적에는 ID와 처분을 남기고, 응답에서 빠졌다는 이유로 해결된 것으로 처리하지 않는다.

큰 계획 개정의 전달량 개선

큰 계획의 일부만 고쳐도 긴 기존 문구를 교체 기준으로 다시 출력하는 비용이 있었다. 기존 계획 해시와 줄 범위를 지정하는 수정 형식을 추가하고, 적용 뒤 같은 최종 계획이 나오는지 확인했다. 해시·범위 검증으로 다른 버전에 잘못 적용되는 수정은 거부한다.

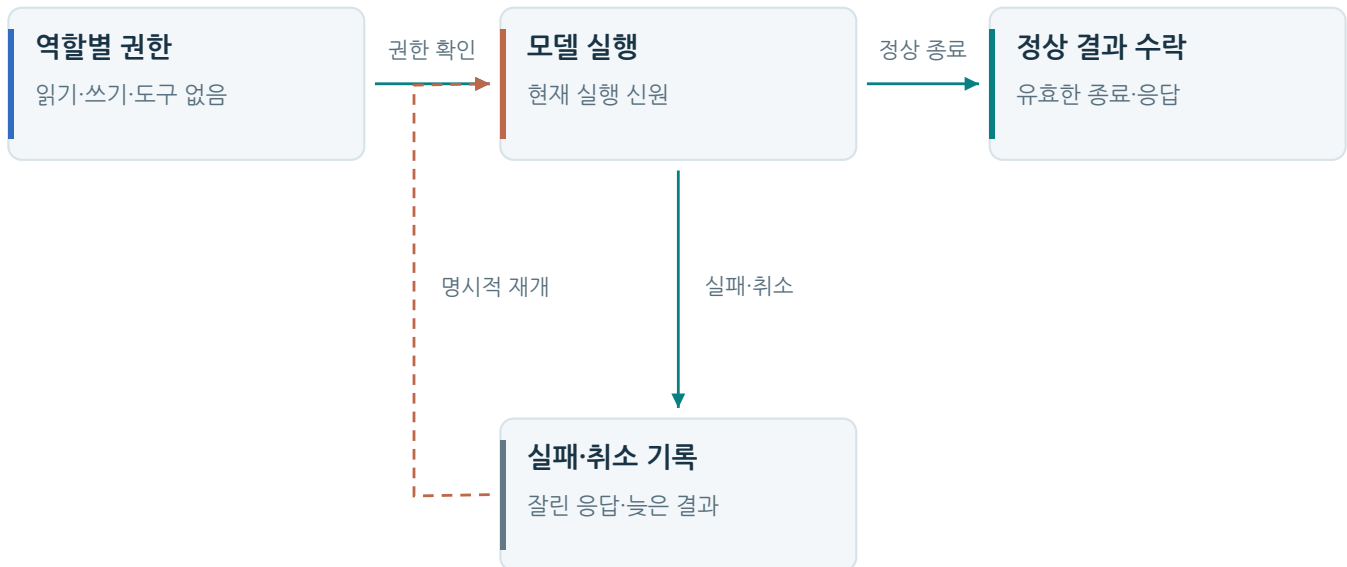
실제 개정의 줄 길이와 동일성을 보존한 익명화 사례에서 수정 데이터는 48,588바이트에서 38,324바이트로 21.1% 줄었다. 줄 번호를 넣는 입력은 1,464바이트 늘었고 최종 계획과 해시는 같았다. 해당 사례의 표현 크기 개선이며 실사용 토큰 절감률은 아니다.

후속 리뷰는 바뀐 지적과 영향받는 호출부를 확인한다. 범위 밖 개선은 근거와 함께 다음 작업으로 넘겨 작은 개선이 현재 리뷰를 끝없이 늘리지 않게 했다.

역할별 권한과 실행 종료

실행 권한·프로세스 종료·결과 형식을 모두 확인한 뒤 결과 수락.

계획·리뷰는 읽기, 구현은 쓰기, 단순 확인 응답은 도구 권한 없이 실행하는 계약을 사용한다. 파일 수정 가능 여부를 특정 공급자 이름에 묶지 않는다. 해당 역할과 실행 도구가 요청한 권한을 지원하는지 먼저 검사한다.



실행 종료와 결과 형식의 확인

모델이 “완료”라고 출력했더라도 프로세스가 정상적으로 끝났는지와 결과 형식이 유효한지를 확인해야 한다. 출력 일부만 읽고 성공을 확정하면 뒤늦게 나온 오류나 잘린 응답을 놓칠 수 있다.

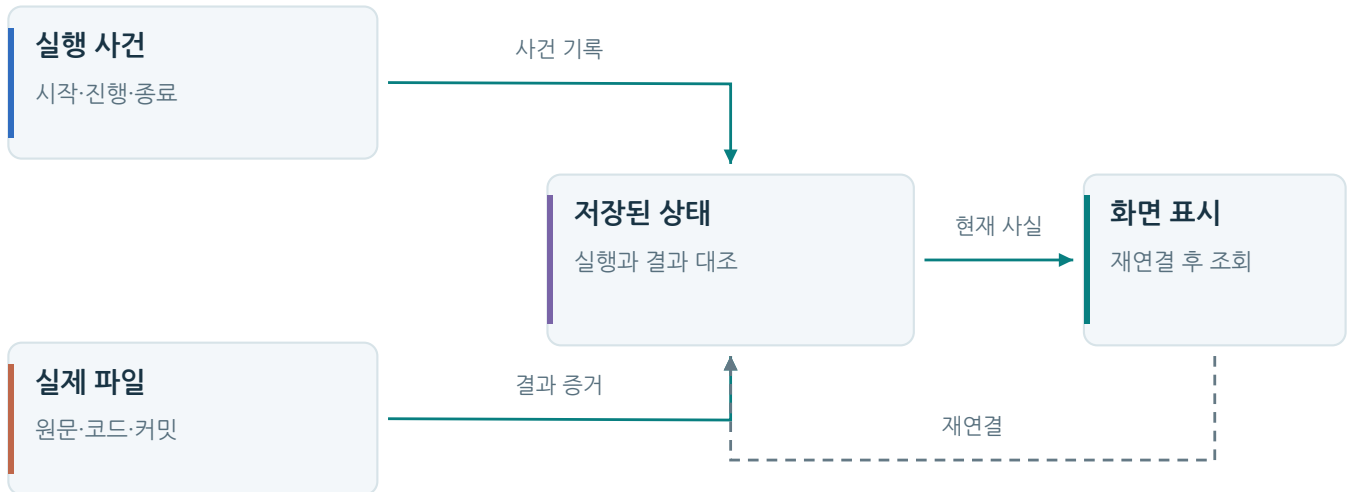
취소한 실행의 응답 차단

취소·재계획·재시작이 일어날 때 실행 신원과 상태를 대조한다. 이전 실행이 늦게 끝나도 최신 계획의 결과나 승인 상태를 덮어쓸 수 없다. 프로세스를 정리할 때도 PID만 보지 않고 시작 신원을 확인해 다른 프로세스를 잘못 종료하지 않도록 한다.

진행 표시와 실제 작업의 구분

실행 시간·문장 증가·완료 선언을 서로 다른 증거로 확인.

계획·리뷰·구현 결과를 파일로 남기면 대화가 길어져도 필요한 원문을 다시 읽을 수 있다. 다만 별칭 파일이 다른 내용으로 교체될 수 있으므로 내용 해시로 고정된 자료를 기준으로 삼는다. DB의 참조와 실제 파일을 읽을 때 해시를 대조한다.



저장된 상태와 재연결

화면은 서버의 실행 기록과 이벤트를 받아 진행 상태를 보여 준다. 실행 시간이나 누적 문장이 늘어난 것만으로 진척을 판단하지 않는다. 현재 턴의 결과·실제 파일·읽은 자료를 대조하며, 재연결 뒤에도 저장된 상태를 확인한다.

검증 근거별 확인 범위

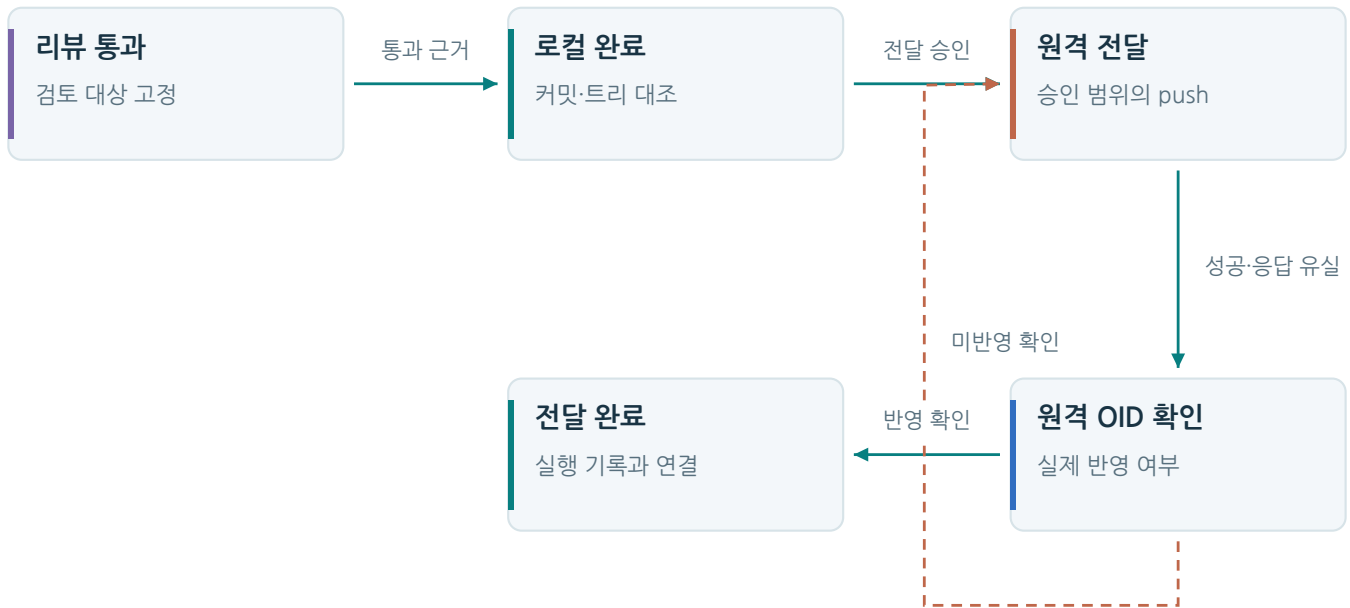
승인한 경로와 변경 규칙을 벗어났는지 확인할 수 있다. Swift 주석·문자열을 구분해야 하는 검사에서는 단순 문자열 치환으로 판단하면 오탐과 누락이 생긴다. 범위 규칙을 적용할 때도 해당 언어의 표현을 고려해야 한다.

서로 다른 증거를 하나의 성공 표시로 합치지 않는 것이 이 설계의 중요한 제한이다.

로컬 완료와 원격 전달

검토한 결과를 로컬 커밋에 연결하고, 원격 반영은 별도 OID로 확인.

단계가 끝나려면 승인된 계획과 리뷰 결과가 실제 로컬 커밋에 연결돼야 한다. 다음 단계는 그 커밋과 결과 묶음을 받는다. 모델이 보고한 커밋 문자열만 믿지 않고 기준 커밋과의 관계, 트리, 작업 상태를 확인한다.



완료 커밋과 단계 통합

갈라진 결과는 충돌 해결과 리뷰 뒤 부모 관계를 보존해 통합한다. 조건부 계획에서 변경이 필요 없었다면, 검토한 기준 커밋을 결과로 사용한다. 허용 경로는 수정 의무가 아니며, 변경이 없는 경우도 현재 승인·검토한 트리·선행 결과를 확인한다.

푸시 응답 유실의 복구

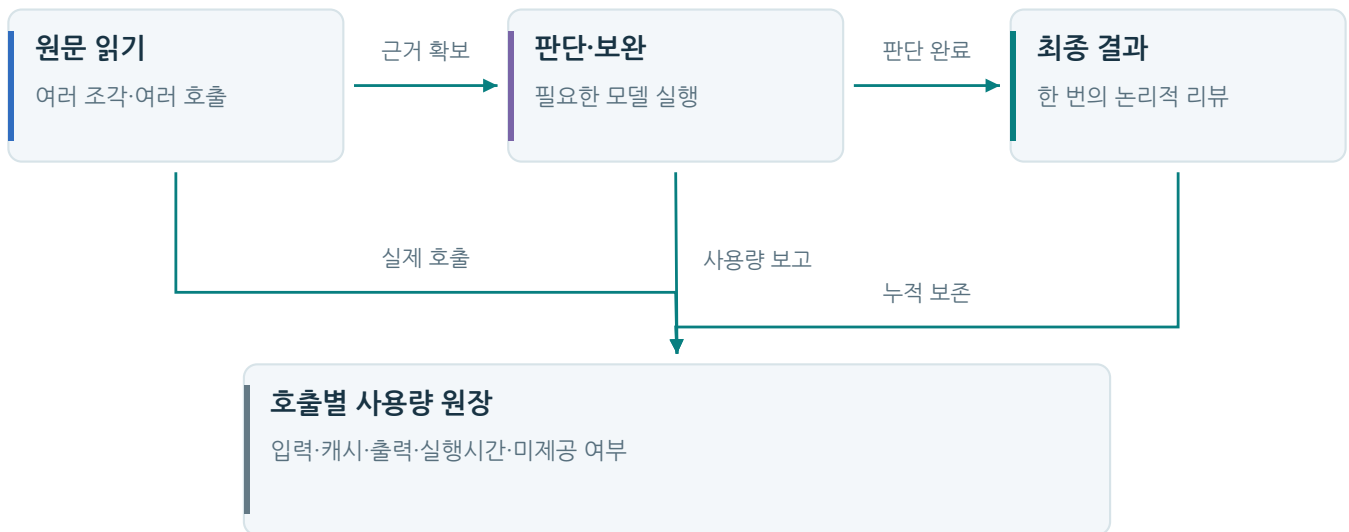
네트워크 오류로 결과를 받지 못해도 원격에는 이미 반영됐을 수 있다. 먼저 원격 커밋을 확인하고 저장된 실행 기록과 대조한다. 이미 반영된 작업을 중복 실행하지 않고 전달 상태를 복구한다.

파일럿에서는 격리된 bare 원격으로 이 흐름을 확인했다. 이 결과가 임의의 실제 서비스 배포와 운영 데이터 변경 까지 검증했다는 뜻은 아니다.

사용량 기록과 실행 한도

원문 읽기 횟수, 논리적 리뷰, 실제 모델 실행을 서로 다른 값으로 기록.

리뷰가 원문을 여러 번 나눠 읽는다고 매번 별개의 리뷰를 시작한 것은 아니다. 읽기와 최종 판단을 하나의 논리적 리뷰로 연결할 수 있어야 큰 자료도 정상적으로 검토할 수 있다. 동시에 실제 모델 호출마다 발생한 사용량과 시간은 빠짐없이 기록해야 한다.



논리적 리뷰와 실제 실행 횟수

수정과 확인 응답이 추가되는 경우에는 해당 정책에 따라 리뷰 예산을 사용한다. 실패한 실행을 성공처럼 지우거나, 새 커밋·새 세션·서버 재시작으로 같은 작업의 사용량을 초기화하지 않는다.

누적 사용량과 적용 한도

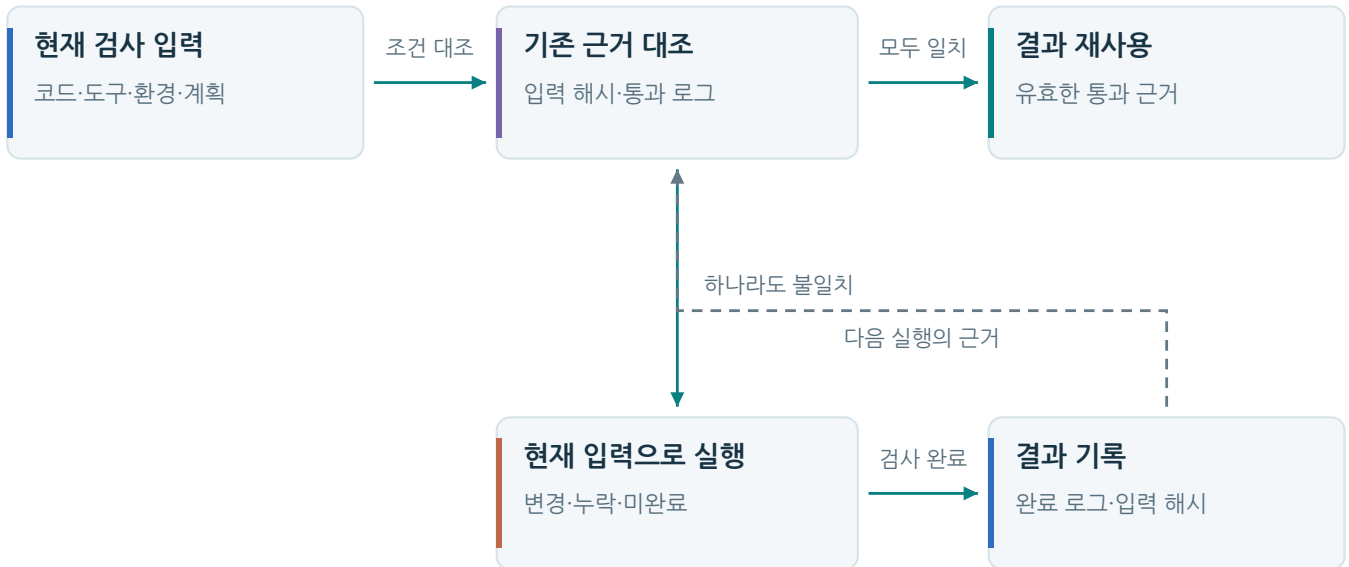
기본 정책은 시간·토큰 사용량을 기록한다. 사용자가 정한 예산·횟수 한도와 공급자의 사용 제한은 따로 적용한다. 중단 이유와 재시도 시점을 보존하며, 취소되거나 오래된 실행의 재시도가 새 계획을 덮어쓰지 못하게 한다.

9월 파일럿에서는 당시 설정에 따라 예산과 리뷰 횟수를 추가했다. 사용량이 누락되면 0으로 채우지 않는다. 현재는 해당 호출과 저장 응답을 보존하며, 미확인 사용량을 인정한 재개 권한도 그 호출에 연결한다.

검사 결과의 재사용 조건

확인된 정적 검사 한 종류의 입력·도구·환경·계획·로그를 함께 대조.

현재 재사용 경로는 알려진 Swift 동시성 정책 정적 검사 한 종류를 대상으로 한다. 관련 입력, 검사 도구, 실행 환경, 승인 계획과 결과 로그를 해시로 확인한다. Git 추적 대상이 아닌 관련 Swift 파일도 입력 집합에 포함해야 한다.



같은 검사를 다시 실행하던 비용

단계가 바뀔 때마다 검사 결과를 다시 구하면 입력과 도구가 같아도 실행 비용이 반복된다. 작업·계획·소스·검사기·실행 환경·로그가 모두 일치하는 결과만 재사용하도록 했다. 단순히 마지막 검사에 성공했다는 표시로는 현재 변경을 검증할 수 없기 때문이다.

저장 실패와 응답 유실의 복구

실제 CLI 프로세스 → 인증 API → DB 경로에서 저장 실패, 응답 유실, 실행 소유권 만료를 재현했다. 재시도할 때 성공 결과를 잃거나 다른 실행의 결과를 채택하지 않는지 확인했다. 입력이 같다는 비교 근거가 없으면 새 검사를 수행한다.

재사용 대상은 Swift 동시성 정책 정적 검사 한 종류다. 컴파일 뒤 WebKit 종료 과정에서 오류가 나타났던 경험도 있어, 정적 검사 성공을 앱 실행·UI 검증으로 확대하지 않았다. 반복 검사를 생각하는 경로는 확인했지만 운영 전체의 시간·토큰 절감량은 아직 측정하지 않았다.

9월 파일럿의 실행 경로

첫 실패는 보존하고, 엔진 중간 수정 없이 새 파일럿에서 다섯 단계와 원격 전달까지 확인.

첫 파일럿은 예산 처리와 계획 확정 경로의 결함으로 중단됐다. 중단 기록을 유지한 채 관련 계약을 수정하고 검토했다. 그 실행을 이어 붙여 성공으로 만들지 않고 새 파일럿에서 전체 흐름을 다시 확인했다.



다섯 단계의 실행 결과

두 번째 파일럿은 실제 Claude·Codex CLI로 수행했다. 진행 중 엔진 파일을 바꾸지 않았고, 비교 대상 243개 파일의 해시가 유지됐다. 마지막에는 모든 단계를 닫고 최종 통합 결과를 격리된 bare 원격에 전달했다.

대상 검사와 실패 검출

최종 대상 프로젝트의 검사 77개가 통과했고, 고의로 필요한 export를 제거한 반대 검사에서는 실패했다. 통과 숫자만으로 판단하지 않고 검사가 실제 결함을 거부하는지도 확인했다.

파일럿 사용량과 검증 범위

9월 파일럿의 실행량과 10월 엔진 후보 검사 결과를 나눠 기록한 검증 사례.

9월 파일럿은 실제 Claude·Codex CLI를 사용해 다섯 단계를 수행하고 격리된 원격에 결과를 전달했다. 아래 수치는 그 실행에서 수집한 관측값이다. 비교 조건이 같은 이전 실행은 없으므로 이 값을 비용이나 시간의 절감률로 해석하지 않는다.

입력 토큰 22,602,633



캐시 입력 18,134,381
전체 입력에 포함

그 밖의 입력 4,468,252

출력 620,113 토큰 · 실행 48회 · 호출 시간 합계 6,566초

캐시 입력과 실행시간의 해석

캐시 입력은 전체 입력에 포함된 값이며 다시 더하지 않는다. 실행시간은 호출별 시간의 합계로, 작업 시작부터 완료까지의 경과 시간과 다르다. 실행 횟수·전달량·비용을 하나의 성능 지표로 합치지 않고 각각의 의미와 함께 보존한다.

공개 코드 검사와 파일럿의 차이

10월 5일 기록한 엔진 후보 검사에서는 개발본 2,417개와 호스트 리뷰 Python 221개가 통과했다. 예상 실패 2개·제외 7개를 별도로 집계했고, TypeScript 검사·웹 빌드·공식 리뷰도 통과했다. 이 결과는 아래 파일럿과 서로 다른 검증 범위를 가진다.

파일럿의 외부 자료는 저장된 시험용 스냅샷이었다. 실제 Slack·Jira·Figma를 모두 연결한 종단 간 검증은 포함하지 않는다. 세션별 읽음 기록은 확인했지만 실제 재전송 전체를 계측하지 않아 “중복 전송 0회”라고 결론 내릴 수 없다. 예산과 리뷰 횟수 추가도 있었으므로 완전 무인 실행으로 소개하지 않는다.

9월 27일 검사: 개발본 1,719 · Python 160 · 공개본 1,381. 새 엔진 검사와 별도 집계.

복구 계약의 검증과 남은 과제

멈춘 지점의 복구부터 다음 단계까지 확인하고, 운영 환경에서의 비교는 별도 수행.

10월 개편의 회귀 검사는 실제 Git과 공개 API를 사용했다. 자료 요청 도중 중지·실패를 만들고 DB를 다시 연 뒤, 승인된 작업의 재개·리뷰·로컬 완료·다음 계획을 연결했다. 모델 응답은 대역이며, 미수신과 접근 오류는 후속 목록에 남는지 확인했다.



복구 이후 실제 진행 확인

운영 서버에는 검토한 코드를 설치하고 기존 계획·체크포인트·배정·한도를 보존한 재시작을 확인했다. 실제 근거 영향 검토도 재개했다. 이 실행의 재개와 대상 앱의 통합 검증 완료는 구분한다.

다음 검증 범위와 설계 기준

이 작업에서 얻은 기준은 AI가 오래 실행됐다는 사실보다 무엇을 읽고, 무엇을 승인받아, 어떤 코드를 검증해 전달했는지 설명할 수 있어야 한다는 것이다. 기능을 추가할 때도 이 연결을 유지하면서 필요한 검증만 확장한다.